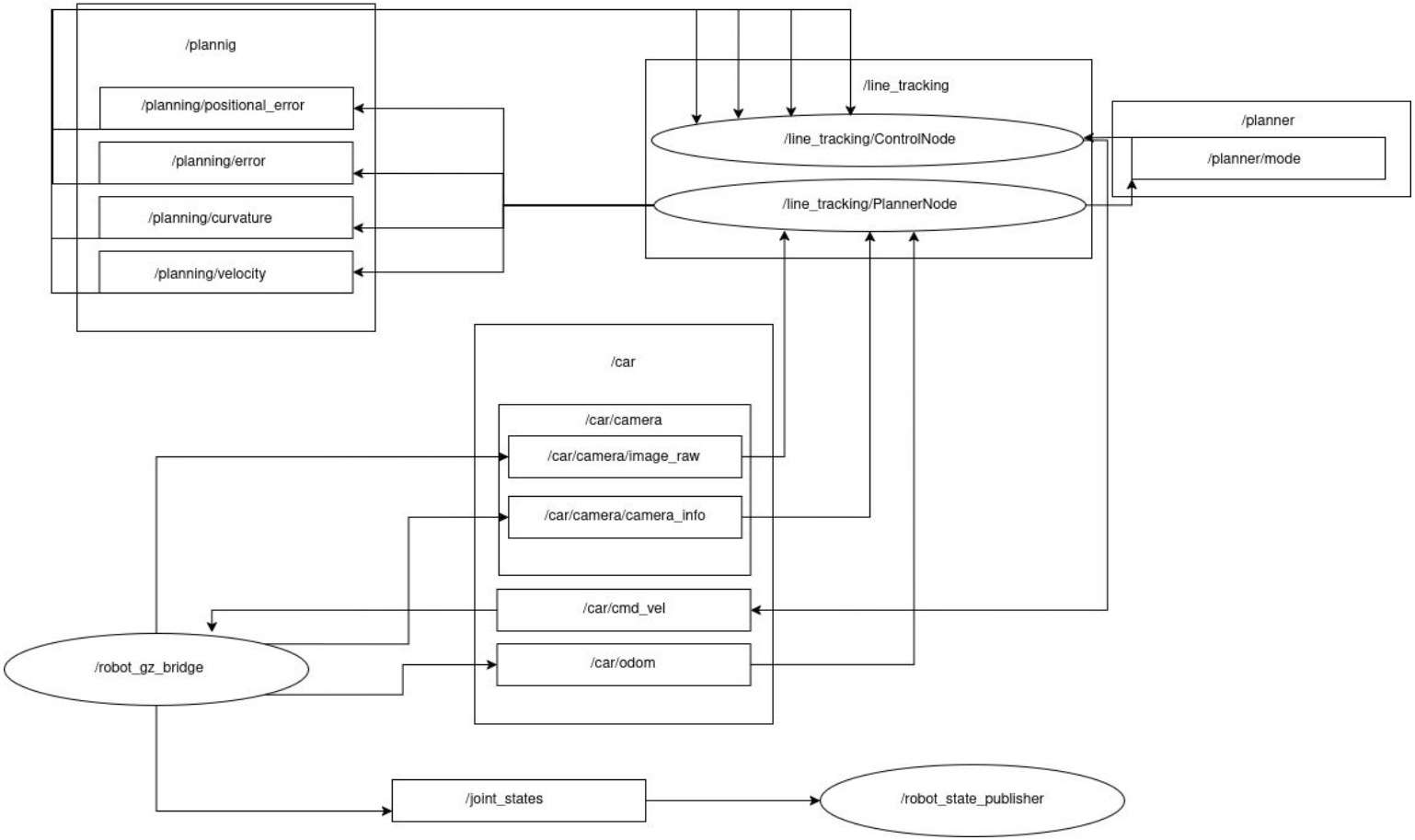
Line Tracking Race

Robotics Exam Project

Francesco Romeo - 634705

2025

General flow



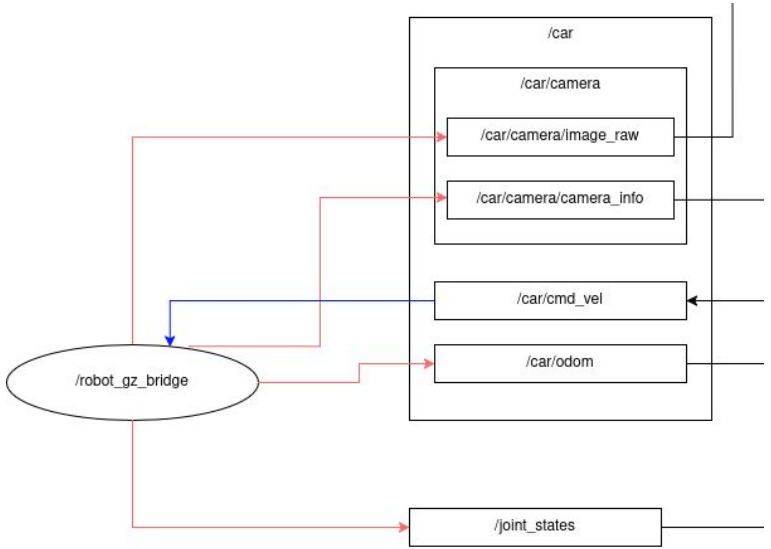
General flow-robot_gz_bridge

Input:

- **Velocity Commands:** geometry_msgs/Twist (topic: /car/cmd_vel)
  Contains linear velocities (x, y, z) and angular velocities (roll, pitch, yaw).

Output:

- **Camera Images:** sensor_msgs/Image (topic: /car/camera/image_raw)
  Provides raw image data captured by the simulated camera.
- **Camera Information:** sensor_msgs/CameraInfo (topic: /car/camera/camera_info)
  Includes both intrinsic and extrinsic camera parameters.
- **Odometry:** nav_msgs/Odometry (topic: /car/odom)
  Represents the estimated position and orientation of the robot.
- **Joint States:** sensor_msgs/msg/JointState (topic: /joint_states)
  Publishes the robot's joint states.



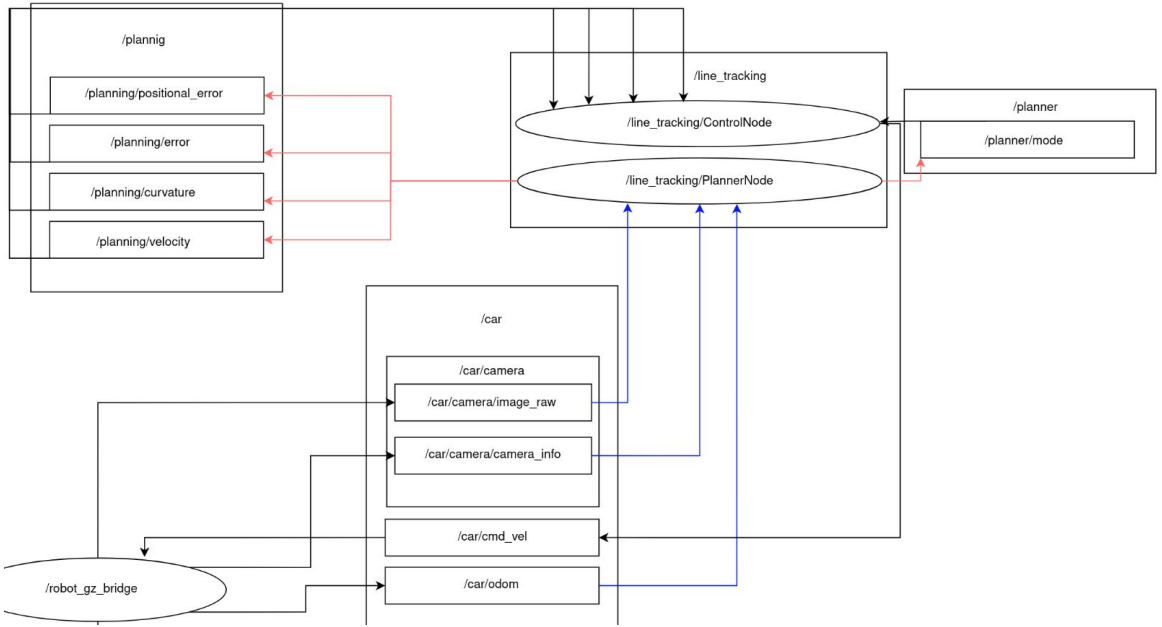
General flow-PlannerNode

Input:

- **Camera Images** — sensor_msgs/Image
  (topic: /car/camera/image_raw — from /car)
- **Camera Information** — sensor_msgs/CameraInfo
  (topic: /car/camera/camera_info — from /car)
- **Odometry** — nav_msgs/Odometry
  (topic: /car/odom — from /car)

Output:

- **Angular Error** — std_msgs/Float32
  (topic: /planning/error)
- **Positional Error** — std_msgs/Float32
  (topic: /planning/positional_error)
- **Curvature** — std_msgs/Float32
  (topic: /planning/curvature)
- **Mode** — std_msgs/Float32
  (topic: /planner/mode)
- **Target Velocity** — std_msgs/Float32
  (topic: /planning/velocity)



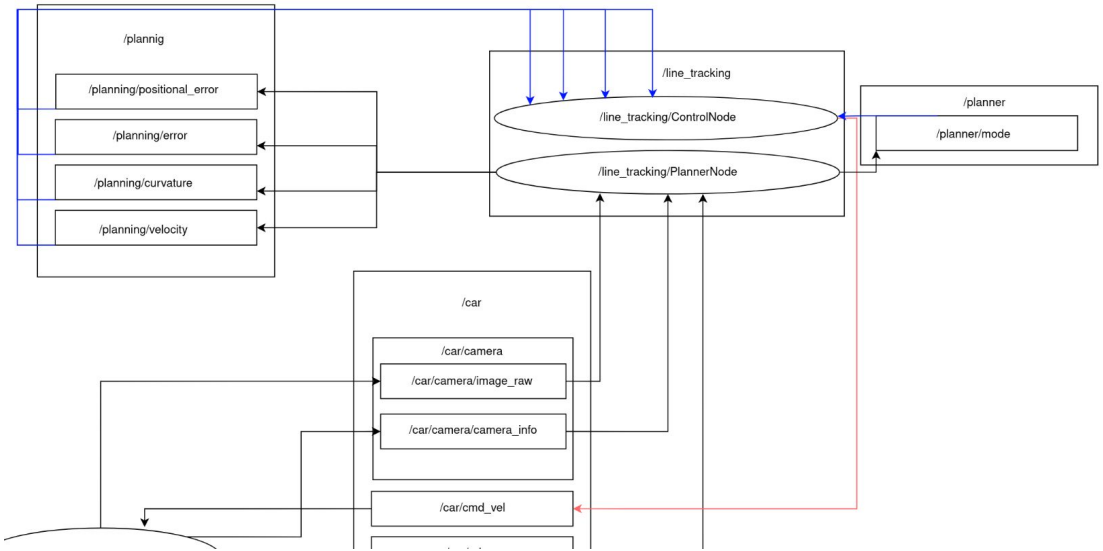
General flow-ControlNode

Input:

- **Angular Error** — std_msgs/Float32
  (topic: /planning/error — from PlannerNode)
- **Positional Error**— std_msgs/Float32
  (topic: /planning/positional_error — from PlannerNode)
- **Curvature** — std_msgs/Float32
  (topic: /planning/curvature — from PlannerNode)
- **Mode** — std_msgs/Float32
  (topic: /planner/mode — from PlannerNode)
- **Target Velocity** — std_msgs/Float32
  (topic: /planning/velocity — from PlannerNode)

Output:

- **Velocity Commands** — geometry_msgs/Twist
  (topic: /car/cmd_vel)

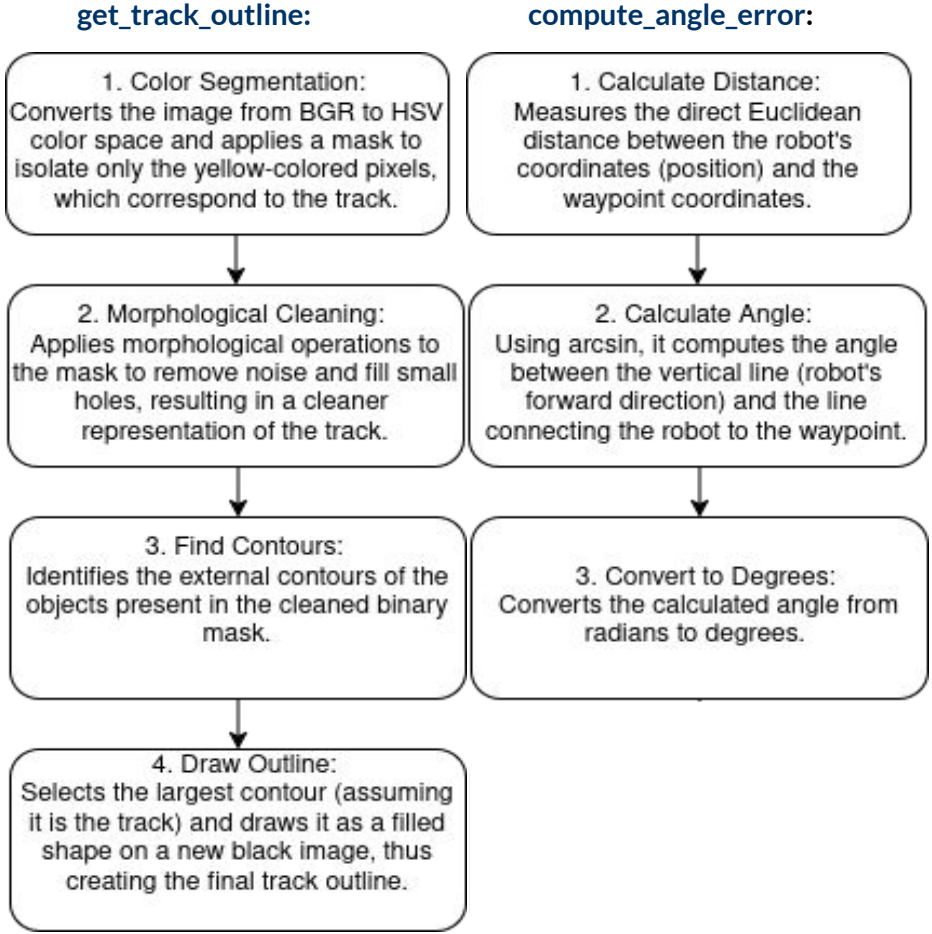


Track outline and angle error

The system first isolates the yellow track from the raw camera feed **get_track_outline**, reducing visual clutter to a usable path mask.

It then extracts the left and right boundaries **extract_track_limits**, a critical step because without solid edge detection you cannot estimate lane width, assess visibility, or derive a trustworthy centerline.

Finally, the robot computes its steering deviation **compute_angle_error** by comparing its estimated pose with the processed track geometry, producing a normalized correction signal that keeps it properly centered.



Track outline and angle error

The visualization window provides a real-time debugging view of the steering logic:

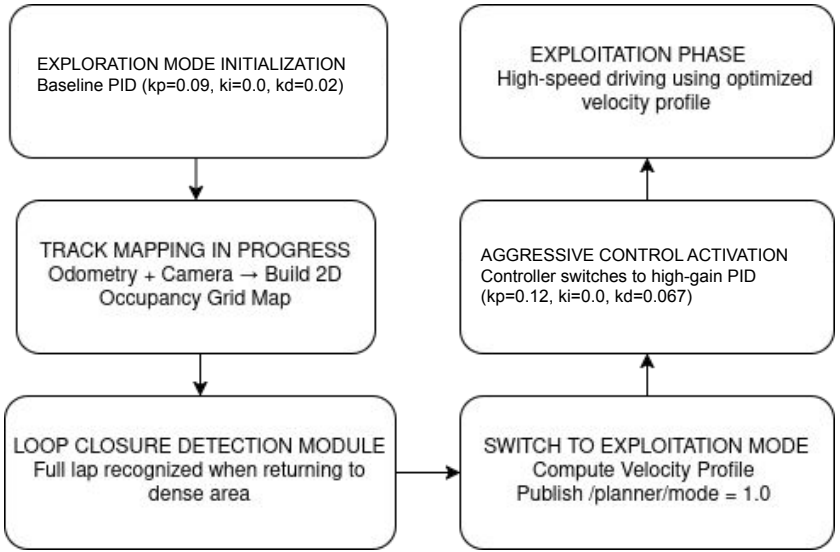
- **Track Mapping:** The **blue and yellow dots** represent the detected track boundaries, while the **magenta dots** mark the calculated centerline ,obtained by interpolating both boundaries and taking their geometric midpoint. This reconstructed path serves as the reference trajectory the robot attempts to follow.
- **Targeting:** The **white circle** indicates the current **target waypoint** on the path ahead and the crosshair (vertical center).
- **Error Calculation:** The two **white lines** represent vectors connecting the robot's position to the screen center and the target waypoint. The **red arc** visually highlights the **angle error**, which is the precise deviation the robot must correct to stay on course.

Exploration vs. Exploitation

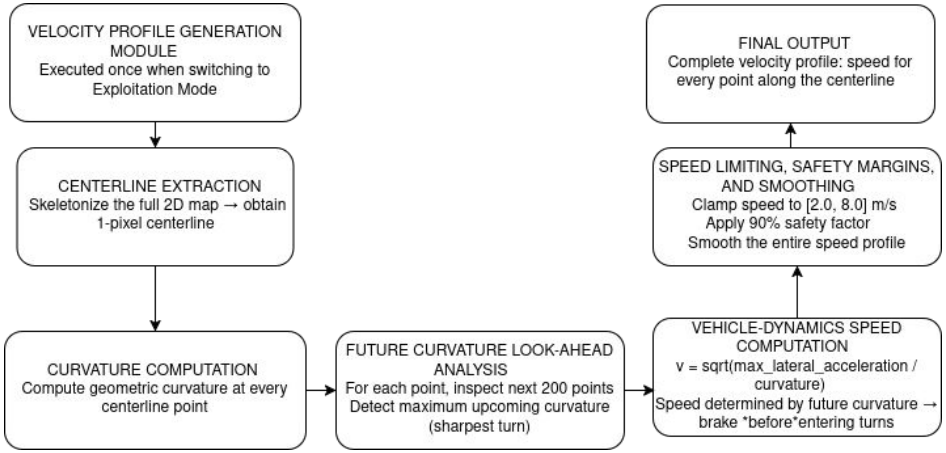
The robot operates using a two-phase strategy designed to balance safety and performance: **Exploration** and **Exploitation**.

Exploration Phase: Initially, the robot begins in Exploration mode, where it cautiously follows the track using conservative PID control gains. During this phase, its primary objective is to construct a comprehensive 2D occupancy map of the circuit by continuously recording the trajectory it has traversed. This process includes detecting track boundaries, updating the map in real-time, and maintaining a reliable estimate of its current pose. Exploration ensures that the robot can safely navigate unknown or partially observed tracks without risking instability at high speeds.

Exploitation Phase: Once the robot identifies that a full lap has been completed, a condition referred to as **loop closure**, it automatically transitions to Exploitation mode. At this point, two critical operations occur: first, the planner computes an optimized velocity profile across the entire mapped track, leveraging curvature analysis and trajectory smoothing to maximize safe speed. Second, the control system switches to a more aggressive set of PID gains, enabling high-speed performance while following the precomputed path.



Computing the Optimal Velocity Profile



Once the track is fully mapped, the **_compute_velocity_profile** function is executed to create the optimal race line and speed strategy.

First, it extracts the precise centerline of the entire track using a skeletonization algorithm. For each point along this centerline, the algorithm computes the **geometric curvature**,

$$\kappa = \frac{|x'\,y'' - y'\,x''|}{(x'^2 + y'^2)^{3/2}}$$

A critical feature of the method is its **lookahead capability**. Rather than reacting solely to the local curvature, the algorithm anticipates upcoming track segments and adjusts the target velocity accordingly.

It applies a **core vehicle dynamics formula**, v=sqrt((μg)/κ) where μ is the friction coefficient, g is gravity, and κ is the curvature, to calculate the **maximum safe speed** for each segment.

This allows the robot to decelerate smoothly before tight corners and accelerate aggressively on straights.

The final output is a comprehensive **velocity profile**: a sequence of points along the track, each associated with a precomputed target speed.

From Camera View to Top-Down Map

- The procedure removes the strong geometric distortion caused by the camera's tilted perspective, which otherwise produces a non-metric, trapezoidal view of the track.
- Inverse Perspective Mapping (IPM) converts this distorted image into a true top-down view of the ground plane, restoring the correct geometric structure.
- A global internal map of the track can only be built after this correction; raw perspective images are unusable for consistent mapping or curvature estimation.
- With the rectified view, the system can accurately extract lane boundaries, estimate curvature, and perform planning tasks that depend on reliable spatial geometry.
- The transformation applies translation, rotation, and pinhole-model projection: ground-plane points are shifted relative to the camera, rotated to compensate for pitch, and finally projected into 2D pixel coordinates.

The PID Control Formula

The better_control_node implements a PD controller with a specific modification to the derivative term for robustness.

Proportional Term (P)

The proportional component is standard:

$$P = K_p\, e(t)$$

Derivative Term (D) with "Dirty Derivative"

A pure derivative is very sensitive to noise in the error signal, which can lead to an unstable and jittery output.

To combat this we use a "Dirty Derivative", which is a low-pass filtered derivative.

The calculation is done in two steps:

1. Discrete Derivative:

$$d_{\text{unf}}(t) = \frac{e(t) - e(t-1)}{\Delta t}$$

2. Low-Pass Filter:

$$d_{\text{filt}}(t) = \alpha\, d_{\text{filt}}(t-1) + (1-\alpha)\, d_{\text{unf}}(t)$$

Final Derivative Term

$$D = K_d\, d_{\text{filt}}(t)$$

The main drawback of this approach is that the low-pass filter introduces a small time lag in the derivative's response.

The PID Control Formula

The I**ntegral (I) term**'s primary purpose is to correct for large, constant steady-state error.
In our implementation, we deliberately omit it because the fast, reactive nature of a PD controller, combined with the low-pass filtered derivative is sufficient for line-following.
While in principle one aims to converge precisely to the setpoint, in practice the robot operates in a continuously changing environment where the error is never truly static.

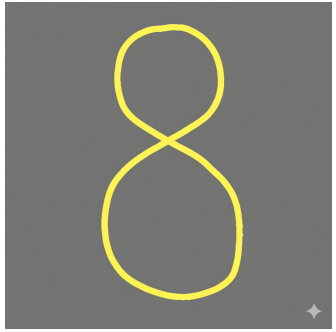
General formula:

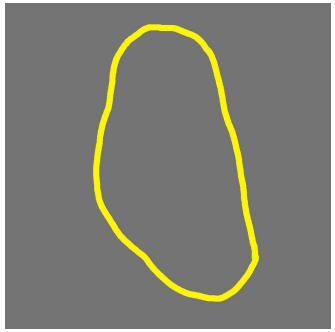
$$u(t) = K_P\, e(t) + K_I \int_0^t e(\tau)\, d\tau + K_D \frac{de(t)}{dt}$$

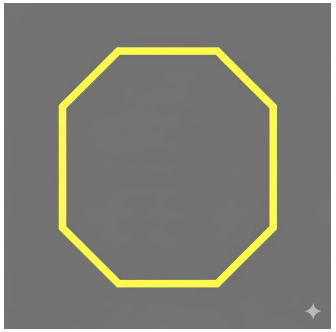
-e(t)is the error at time t.
-Kp (Proportional Gain): Reacts to the present error.
-Ki(Integral Gain): Accumulates past errors to eliminate steady-state error.
-Kd (Derivative Gain): Predicts future error based on its current rate of change.

Validation on Variable Geometries

A central strength of this project is the algorithm's demonstrated robustness across a wide variety of track geometries and scales.
The system has been tested on circuits ranging from tight, small-scale polygons to larger circular tracks, including configurations with self-crossing segments.
In all cases, the controller successfully maintained precise line-following performance, continuously adjusting steering and velocity to accommodate the curvature and layout of the track.
The filtered derivative term and adaptive control gains contribute to stability and responsiveness, allowing the vehicle to react rapidly without introducing oscillations or excessive overshoot.
The following laps provide a visual illustration of the algorithm's behavior, showing how trajectories and velocity profiles naturally vary according to each track's specific geometry.







These visualizations emphasize the algorithm's flexibility and its ability to generate control actions that are tailored to the current conditions of the circuit, resulting in consistently reliable performance across tracks of different shapes and scales.

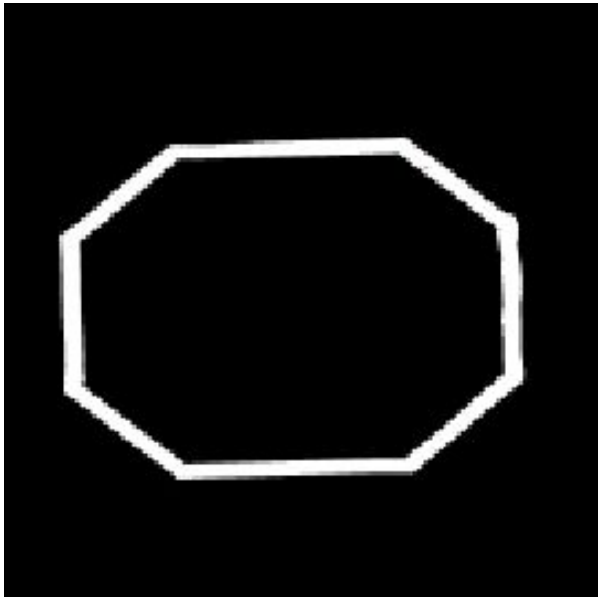
Control Error vs Positional Error

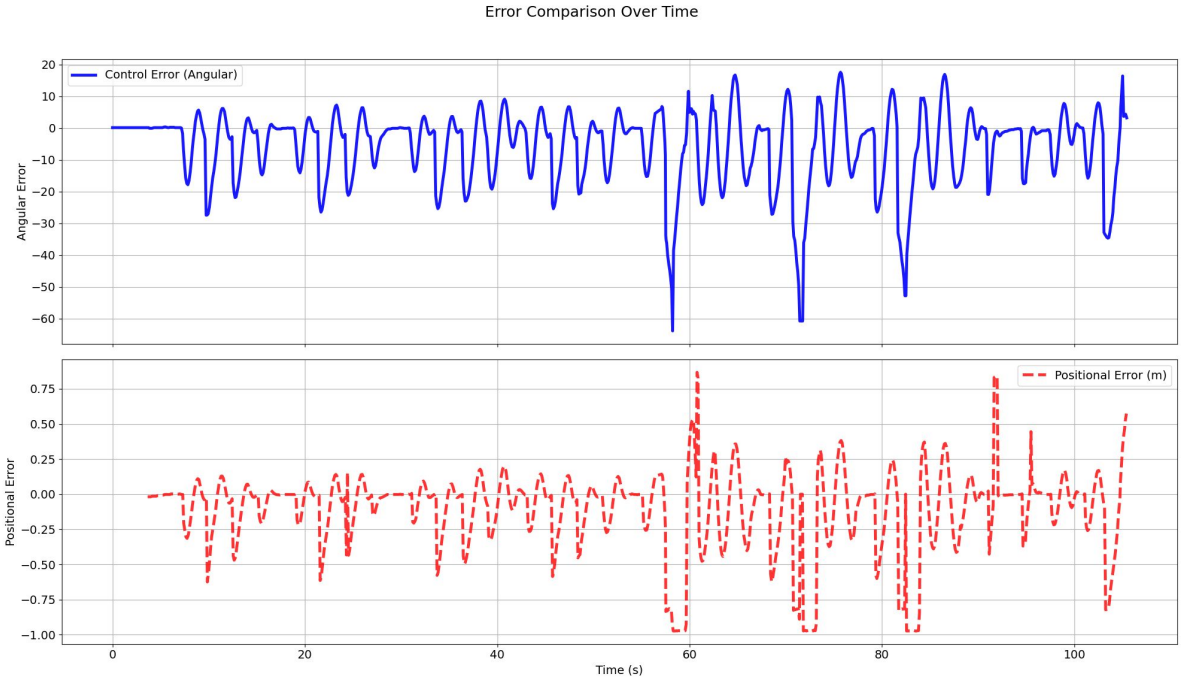
The **Control Error** represents the angular deviation toward a future waypoint and essentially defines the direction the robot aim for.
Because it looks ahead rather than reacting to momentary disturbances, it produces a naturally smooth and reliable signal.
This anticipatory quality makes it the backbone of the steering logic: it filters out irrelevant jitter, stabilizes the control loop, and allows the robot to maintain a confident trajectory even as speed increases.

The **Positional Error** shows the robot's immediate lateral offset, reacting to micro-movements and noise.
Peaks occur near curves, likely because the robot cuts slightly, but are controlled at low speed.
At higher speeds, curves are taken wider, causing slight drift while still staying on track.
This effect is more pronounced on an octagonal track with sharp corners.
This behavior comes from the kinematic model, which ignores friction and centripetal forces, so high-speed slips violate its assumptions.

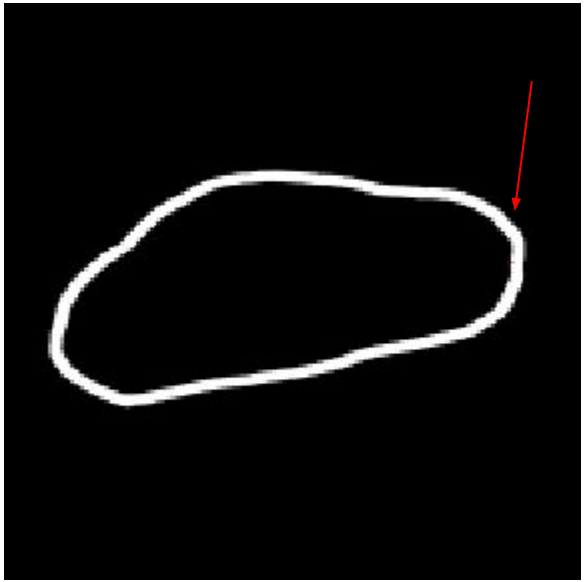
Note: at ~60 seconds, there is a switch in modality.

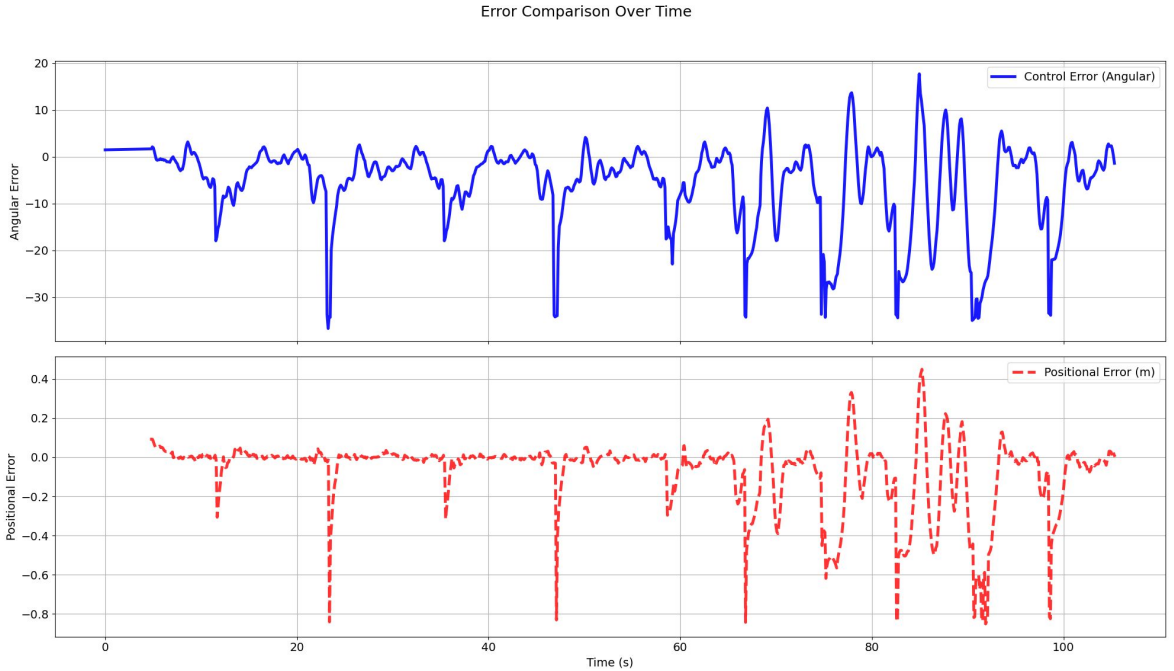
Track 1



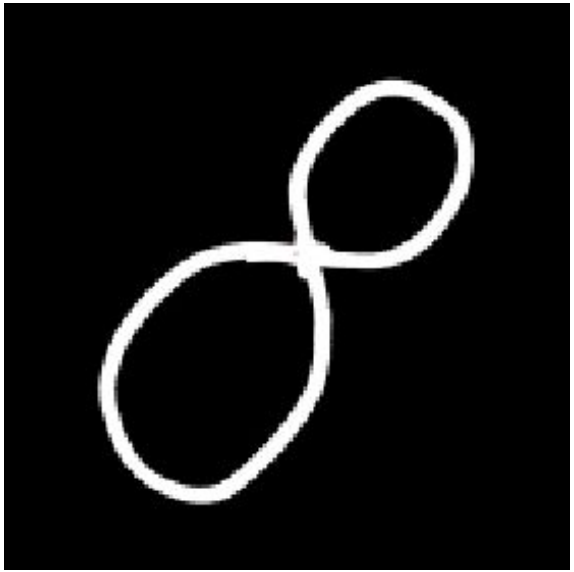

Error Comparison Over Time
Control Error (Angular)
Angular Error
20
10
0
−10
−20
−30
−40
−50
−60
Positional Error (m)
Positional Error
0.75
0.50
0.25
0.00
−0.25
−0.50
−0.75
−1.00
0
20
40
60
80
100
Time (s)

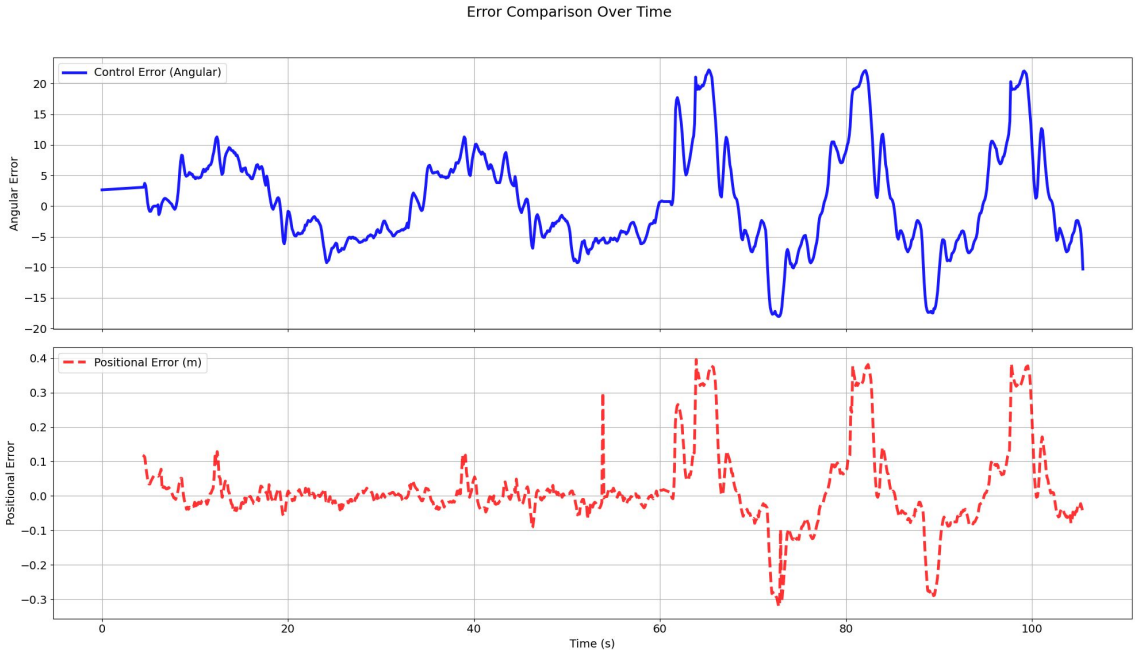
Track 2




Error Comparison Over Time
Control Error (Angular)
Angular Error
20
10
0
−10
−20
−30
Positional Error (m)
Positional Error
0.4
0.2
0.0
−0.2
−0.4
−0.6
−0.8
0
20
40
60
80
100
Time (s)

Track 3




Error Comparison Over Time
Control Error (Angular)
Angular Error
20
15
10
5
0
−5
−10
−15
−20
Positional Error (m)
Positional Error
0.4
0.3
0.2
0.1
0.0
−0.1
−0.2
−0.3
0
20
40
60
80
100
Time (s)

Demo

Questions?

Thank you.